

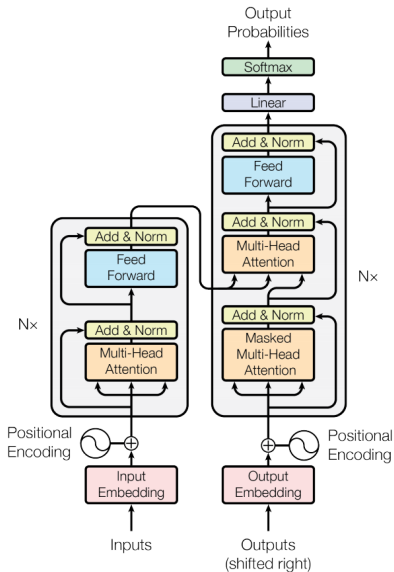
Deep Learning

Vazgen Mikayelyan

June 22, 2021



Transformer



Self Attention Layer

- This layer aims to encode a word based on all other words in the sequence. It measures the encoding of the word against the encoding of another word and gives a new encoding.

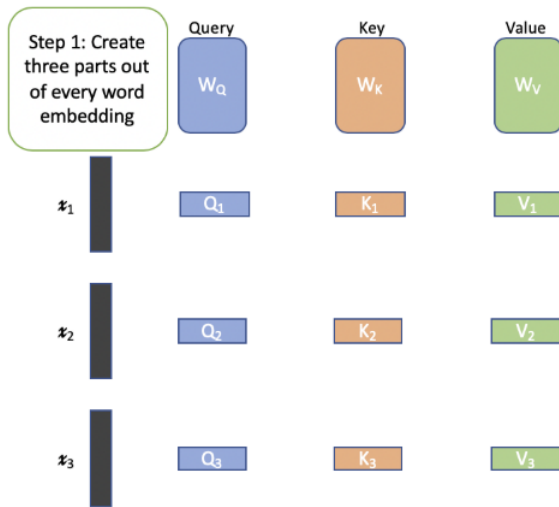
Self Attention Layer

- This layer aims to encode a word based on all other words in the sequence. It measures the encoding of the word against the encoding of another word and gives a new encoding.
- Given an embedding x , it learns three separate smaller embeddings from it — query, key and value.

Self Attention Layer

- This layer aims to encode a word based on all other words in the sequence. It measures the encoding of the word against the encoding of another word and gives a new encoding.
- Given an embedding x , it learns three separate smaller embeddings from it — query, key and value.
- During the training phase, the W_q , W_k , and W_v matrices are learnt to get the query, key and value embeddings.

Self Attention Layer



Self Attention Layer

- Say x_1 wants to know its value with respect to x_2 . So it will 'query' x_2 .

Self Attention Layer

- Say x_1 wants to know its value with respect to x_2 . So it will 'query' x_2 .
- x_2 will provide the answer in the form of its own 'key', which can then be used to get a score representing how much it values x_1 by taking a dot product with the query. Since both have the same size, this will be a single number.

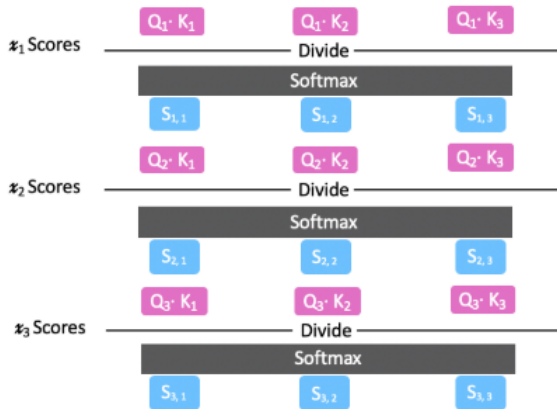
Self Attention Layer

- Say x_1 wants to know its value with respect to x_2 . So it will 'query' x_2 .
- x_2 will provide the answer in the form of its own 'key', which can then be used to get a score representing how much it values x_1 by taking a dot product with the query. Since both have the same size, this will be a single number.
- Then x_1 will take all these scores and perform softmax.

Self Attention Layer

- Say x_1 wants to know its value with respect to x_2 . So it will 'query' x_2 .
- x_2 will provide the answer in the form of its own 'key', which can then be used to get a score representing how much it values x_1 by taking a dot product with the query. Since both have the same size, this will be a single number.
- Then x_1 will take all these scores and perform softmax.
- This step will be performed with every word.

Self Attention Layer



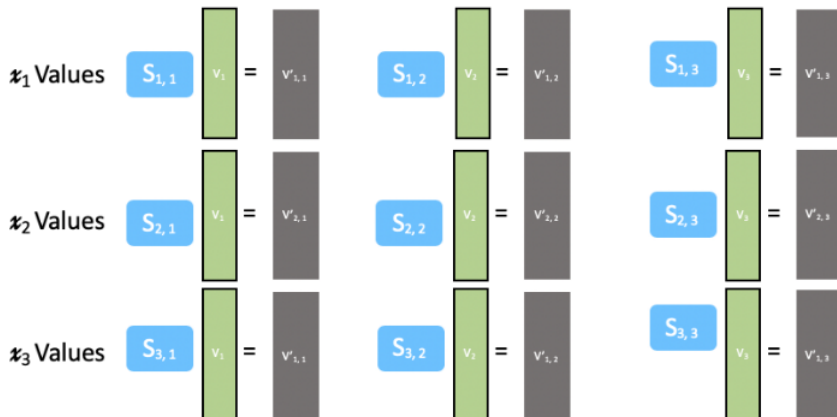
Self Attention Layer

- x_1 will now use this score and the 'value' of the corresponding word to get a new value of itself with respect to that word.

Self Attention Layer

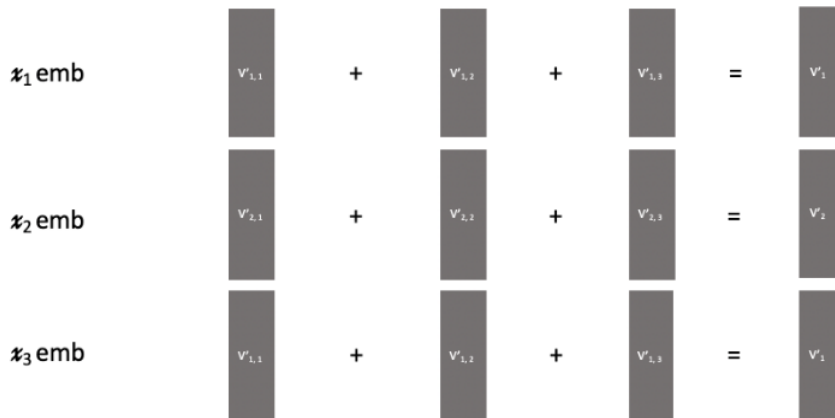
- x_1 will now use this score and the 'value' of the corresponding word to get a new value of itself with respect to that word.
- If the word is not relevant to x_1 then the score will be small and the corresponding value will be reduced a factor of that score and similarly the significant words will get their values bolstered by the score.

Self Attention Layer



Self Attention Layer

Finally, the word x_1 will create a new 'value' for itself by summing up the values received. This will be the new embedding of the word.



$$\text{Attention}(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

where

- inputs: a query q and a set of key-value (K-V) pairs to an output,

$$\text{Attention}(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

where

- inputs: a query q and a set of key-value (K-V) pairs to an output,
- query, keys, values and output are all vectors,

$$\text{Attention}(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

where

- inputs: a query q and a set of key-value (K-V) pairs to an output,
- query, keys, values and output are all vectors,
- output is a convex combination of values,

$$\text{Attention}(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

where

- inputs: a query q and a set of key-value (K-V) pairs to an output,
- query, keys, values and output are all vectors,
- output is a convex combination of values,
- weight of each value is computed by an inner product of query and corresponding key,

$$\text{Attention}(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

where

- inputs: a query q and a set of key-value (K-V) pairs to an output,
- query, keys, values and output are all vectors,
- output is a convex combination of values,
- weight of each value is computed by an inner product of query and corresponding key,
- queries and keys have the same dimensionality d_k , values have d_v .

When we have multiple queries q , we stack them in a matrix Q :

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(QK^T\right) V.$$

When we have multiple queries q , we stack them in a matrix Q :

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(QK^T\right) V.$$

- Problem: as d_k gets large, the variance of $q^T k$ increases, thus some values inside the softmax gets large, hence its gradients gets smaller.

When we have multiple queries q , we stack them in a matrix Q :

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(QK^T\right) V.$$

- Problem: as d_k gets large, the variance of $q^T k$ increases, thus some values inside the softmax gets large, hence its gradients gets smaller.
- Solution:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Multi-Head Attention Layer

Multihead attention assumes that all inputs and outputs have the same length d_{model} . If inputs hasn't length d_{model} , we pass it through one fully connected layer.

Multi-Head Attention Layer

Multihead attention assumes that all inputs and outputs have the same length d_{model} . If inputs hasn't length d_{model} , we pass it through one fully connected layer.

$$\text{Multihead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where

$$\text{head}_i = \text{Attention}(xW_i^Q, xW_i^K, xW_i^V)$$

$$W_i^Q \in \mathbb{R}^{d_{model} \times d_k}, W_i^K \in \mathbb{R}^{d_{model} \times d_k}, W_i^V \in \mathbb{R}^{d_{model} \times d_v}, W^O \in \mathbb{R}^{hd_v \times d_{model}}$$

Masked Multi-Head Attention Layer

- At any position, a word may depend on both the words before it as well as the ones after it.

Masked Multi-Head Attention Layer

- At any position, a word may depend on both the words before it as well as the ones after it.
- This is why in the self-attention layer, the query was performed with all words against all words.

Masked Multi-Head Attention Layer

- At any position, a word may depend on both the words before it as well as the ones after it.
- This is why in the self-attention layer, the query was performed with all words against all words.
- But at the time of decoding, when trying to predict the next word in the sentence, logically, it should not know what are the words which are present after the word we are trying to predict.

Masked Multi-Head Attention Layer

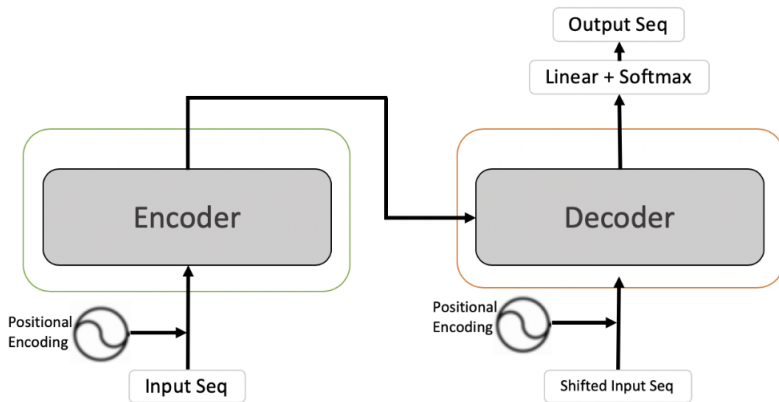
- At any position, a word may depend on both the words before it as well as the ones after it.
- This is why in the self-attention layer, the query was performed with all words against all words.
- But at the time of decoding, when trying to predict the next word in the sentence, logically, it should not know what are the words which are present after the word we are trying to predict.
- This is why the embeddings for all these are masked by multiplying with 0.

Feed-Forward Network

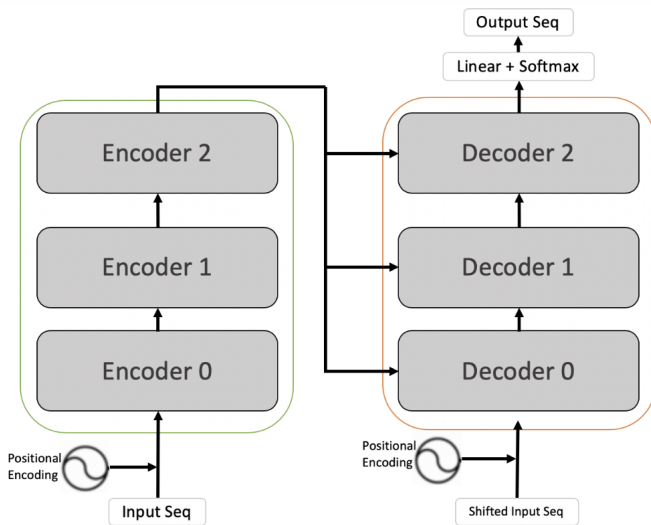
this part is a position free neural network, which consists of two fully connected layers with a ReLU activation in between:

$$\text{FFN}(x) = W_2 \cdot \text{ReLU}(W_1 x + b_1) + b_2$$

Encoder-Decoder Architecture



Encoder-Decoder Architecture



Encoder-Decoder Architecture

Queries

Decoder Input

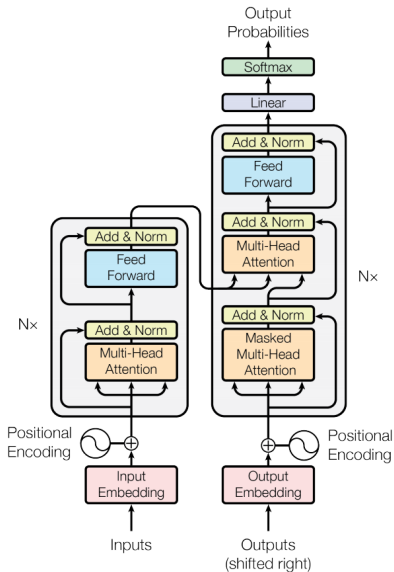
Keys

Encoder output

Values

Encoder output

Transformer



1 Dilated and Transposed Convolutions

Definition 1

Let $F : \mathbb{Z}^2 \rightarrow \mathbb{R}$ be a discrete function. Let $\Omega_r = [-r, r]^2 \cap \mathbb{Z}^2$ and let $k : \Omega_r \rightarrow \mathbb{R}$ be a discrete filter of size $(2r + 1)^2$. The discrete convolution operator $*$ can be defined as

$$(F * k)(p) = \sum_{s+t=p} F(s) k(t)$$

Definition 1

Let $F : \mathbb{Z}^2 \rightarrow \mathbb{R}$ be a discrete function. Let $\Omega_r = [-r, r]^2 \cap \mathbb{Z}^2$ and let $k : \Omega_r \rightarrow \mathbb{R}$ be a discrete filter of size $(2r + 1)^2$. The discrete convolution operator $*$ can be defined as

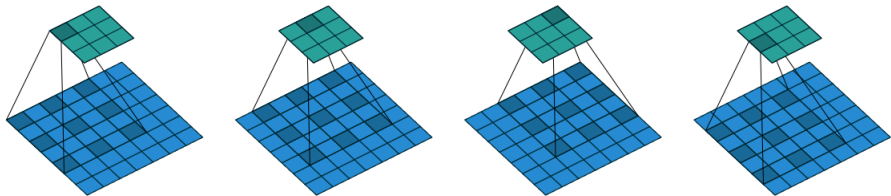
$$(F * k)(p) = \sum_{s+t=p} F(s) k(t)$$

Definition 2

Let $F : \mathbb{Z}^2 \rightarrow \mathbb{R}$ be a discrete function. Let $\Omega_r = [-r, r]^2 \cap \mathbb{Z}^2$ and let $k : \Omega_r \rightarrow \mathbb{R}$ be a discrete filter of size $(2r + 1)^2$. The discrete l -dilated convolution operator $*_l$ can be defined as

$$(F *_l k)(p) = \sum_{s+lt=p} F(s) k(t)$$

Dilated/Atrous Convolution



Dilated/Atrous Convolution

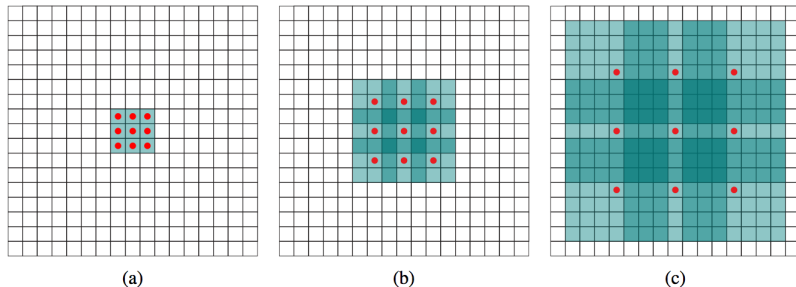


Figure 1: Systematic dilation supports exponential expansion of the receptive field without loss of resolution or coverage. (a) F_1 is produced from F_0 by a 1-dilated convolution; each element in F_1 has a receptive field of 3×3 . (b) F_2 is produced from F_1 by a 2-dilated convolution; each element in F_2 has a receptive field of 7×7 . (c) F_3 is produced from F_2 by a 4-dilated convolution; each element in F_3 has a receptive field of 15×15 . The number of parameters associated with each layer is identical. The receptive field grows exponentially while the number of parameters grows linearly.

1D Dilated Convolution

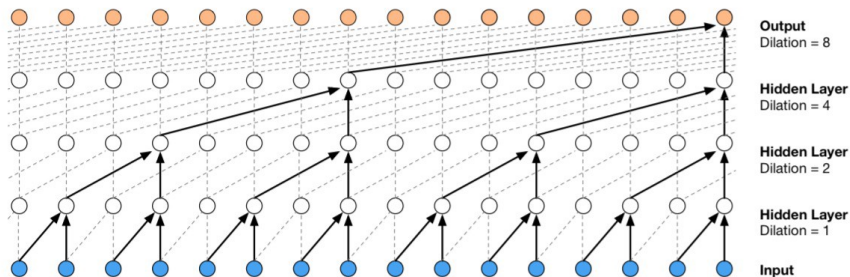
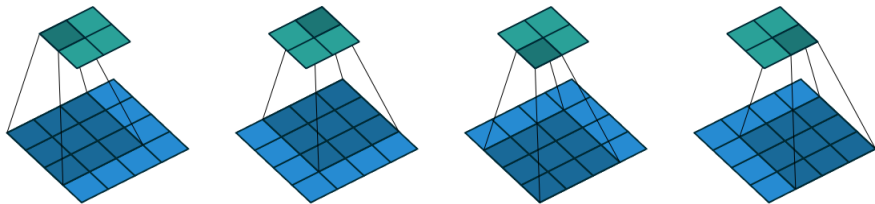
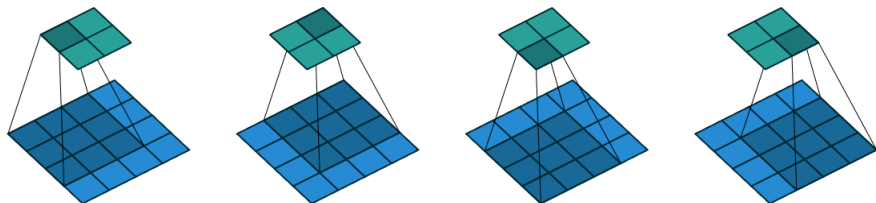


Figure 3: Visualization of a stack of *dilated* causal convolutional layers.

Convolution as a Matrix Operation



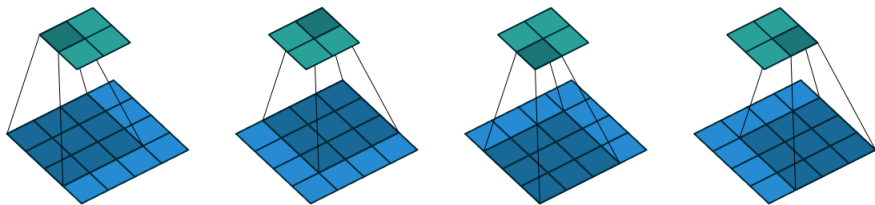
Convolution as a Matrix Operation



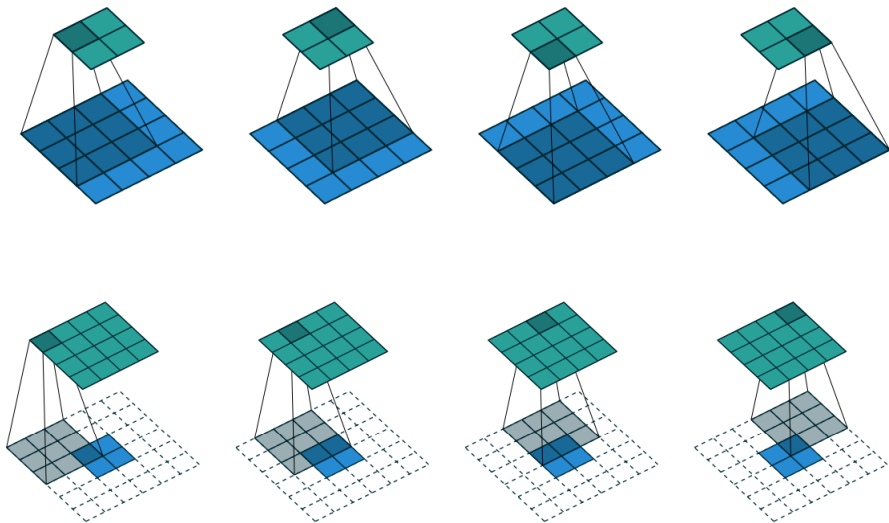
$$\begin{pmatrix} w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 & 0 & 0 & 0 & 0 \\ 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 \\ 0 & 0 & 0 & 0 & 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} \end{pmatrix}$$

This linear operation takes the input matrix flattened as a 16-dimensional vector and produces a 4-dimensional vector that is later reshaped as the 2×2 output matrix.

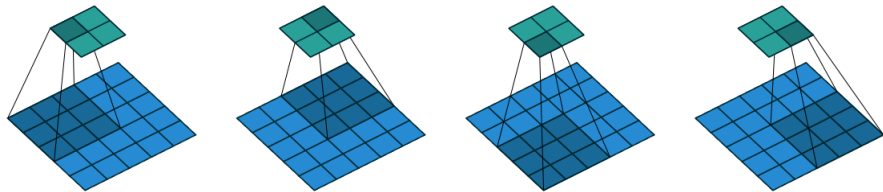
Transposed Convolution (stride=0)



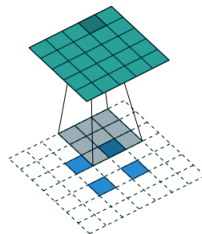
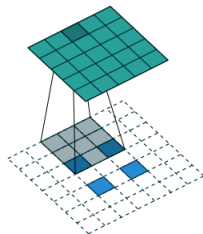
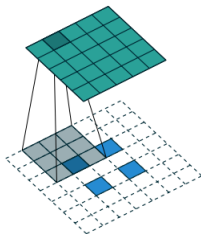
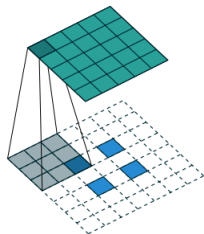
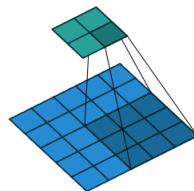
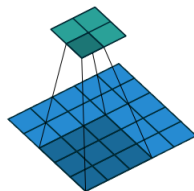
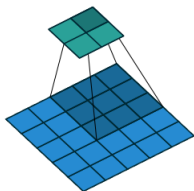
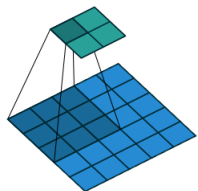
Transposed Convolution (stride=0)



Transposed Convolution (stride=1)



Transposed Convolution (stride=1)



UNet

